Defect Log

UNZA Hostel Maintenance Request and Tracking System

University of Zambia — Final Year Project

Defect Log

Construction-phase deliverable (UP): every genuine defect found during this project's development, with its root cause, its fix, and how the fix was verified — not a hypothetical exercise, an actual record kept as the system was built. (A separate exercise — deliberately breaking two business rules to confirm the test suite actually catches them — is a *testing technique*, not a defect, and is documented in TESTING.md §3 instead.)

Severity: Critical (data loss/security) · High (a real feature broken in production) · Medium (misleading/incorrect behaviour, no data at risk) · Low (cosmetic or documentation only).

ID	Severity	Found during	Summary
DEF-01	High	Construction	Ambiguous foreign-key relationship broke request/upvote queries after a new migration
DEF-02	Low	Construction	Documentation files corrupted by wrong text encoding from a helper script
DEF-03	Medium	Transition	Offline error message misleadingly blamed "the server"
DEF-04	High	Transition	A brief network drop silently signed the user out
DEF-05	Low	Transition	Placeholder text in Markdown docs silently stripped by renderers
DEF-06	Low	Transition	A preventive-maintenance job's "Reported By" section misattributed it to an administrator
DEF-07	Medium	Transition	Technicians received no announcements at all
DEF-08	Low	Transition	A recurring-maintenance job's description doubly stated it was preventive, once a badge was added

---

DEF-01 — Ambiguous foreign-key relationship after adding upvotes

Severity: High. **Found:** Construction, via a live-environment error screenshot from manual testing, shortly after migration `009` (request upvotes) was added.

Symptom: Supabase's embed query resolver returned *"more than one relationship was found for 'maintenance_requests' and 'users'"* whenever a request row tried to join its reporting student, breaking `GET /requests` and the request detail view.

Root cause: `request_upvotes` added a second foreign key path between `maintenance_requests` and `users` (via the upvoting user), so Supabase's embed syntax could no longer infer *which* relationship `student:users(...)` meant without help — there were now two valid candidates where there used to be one.

Fix: Added explicit `!student_id` relationship hints to every affected embed in `requests.repository.js` and `activity.repository.js`, so the query names the exact foreign key to follow rather than leaving Supabase to guess.

Verification: Reproduced against the live database before the fix, confirmed resolved against the live database after it, then committed separately (so the fix could be reverted independently of the feature that exposed it) and covered going forward by the integration tests that already exercise `GET /requests` and request-detail reads.

DEF-02 — Text-encoding corruption from a helper script

Severity: Low (documentation only, never shipped in the application). **Found:** Construction, visually, while reviewing generated documentation.

Root cause: A Python script used to batch-edit documentation wrote files without explicitly specifying UTF-8 output encoding, corrupting em-dash characters into mojibake on a Windows default codepage.

Fix: Rewrote the affected files directly with correct UTF-8 encoding; stopped using implicit-encoding scripts for text edits afterward in favour of the editor's own file tools.

Verification: Visual re-inspection of every affected file.

DEF-03 — Offline error message misleadingly blamed "the server"

Severity: Medium (a real, reported UX defect, not a data-integrity issue). **Found:** Transition, reported directly by the project's own end-user tester while using the deployed app.

Symptom: A student on working-but-unreliable hostel Wi-Fi, whose request could not reach the API, saw "*Could not reach the server. It may be temporarily unavailable*" — confidently blaming the server for what was actually a connectivity problem.

Root cause: `navigator.onLine` reflects only whether the network *adapter* believes it has a link, not whether that link actually reaches the internet — a well-known limitation of the API. The client's error handler trusted `navigator.onLine === false` as the *only* signal for "offline", so a connected-but-non-functional Wi-Fi link (the common case on weak hostel Wi-Fi) fell through to the server-blaming message.

Fix: Reworded the ambiguous case (`onLine !== false` , i.e. "not sure") to lead with "check your internet connection" rather than confidently naming the server, while keeping the more specific "you're offline" message for the one case that *is* certain (`onLine === false`). See `client/src/services/api.js` .

Verification: Manual testing with Wi-Fi connected but internet disabled (router unplugged from its uplink) versus the adapter fully disabled, confirming each path now shows the intended message.

DEF-04 — A brief network drop silently signed the user out

Severity: High (repeatedly logs a legitimate user out of a working session; found as the root cause behind DEF-03's user report, not a separate report). **Found:** Transition, during investigation of DEF-03.

Symptom: Opening the installed app while briefly offline (e.g. right after unlocking a phone, before Wi-Fi reconnects) would clear a perfectly valid session and land the user back on the Login screen — the *actual* underlying cause of what looked, to the user, like "the site is broken".

Root cause: `AuthProvider`'s session-check on app load ran `getCurrentUser().catch(clearSession)` — **any** rejection, including a pure network failure with no bearing on whether the token was still valid, cleared the stored session.

Fix: The catch handler now distinguishes a network failure (`err.isNetworkError`, already flagged by `api.js`) from a genuine authentication rejection, and only clears the session on the latter. A failed check additionally now retries automatically on the browser's `online` event, and `Login.jsx` redirects away once that retry restores the session — so a user is never stuck re-entering a password for a problem that was never about their password.

Verification: Manual testing: go offline before reload, confirm the existing session survives and the app signs back in automatically once connectivity returns, with no re-entered credentials.

DEF-05 — Placeholder text silently stripped from Markdown documentation

Severity: Low (documentation rendering only). **Found:** Transition, visually, while spot-checking a regenerated PDF.

Symptom: `**Already reported in <your hostel>**` rendered as "*Already reported in .*" — the placeholder text vanished entirely — in both the PDF export and (independently confirmed) on GitHub's own Markdown rendering.

Root cause: Both renderers (the `marked` library used for PDF generation, and GitHub's) treat `<word word>` outside a code span as an unrecognised HTML tag and silently drop it, rather than treating it as literal text.

Fix: HTML-escaped every instance of this pattern found outside a backtick code span (`<your hostel>`), after a systematic grep across every documentation file confirmed which instances were actually at risk (ones already inside `` `backticks` `` render as literal text safely and did not need changing).

Verification: Regenerated the affected PDFs and visually confirmed the placeholder text now renders correctly.

DEF-06 — A preventive-maintenance job's "Reported By" section named an administrator

Severity: Low (misleading display, not incorrect data — the underlying `student_id` value is correct by design; see DOMAIN_MODEL.md §3). **Found:** Transition, during a feature-clarity request (a preventive job

needed to read more clearly as machine-generated, not student-reported).

Symptom: A technician opening a preventive-maintenance-generated task saw "Reported By: Dr. Mwamba, Student ID: STF-0001" — technically following from how the row is stored (the schedule's owning administrator is the `student_id` on the generated request), but confusing to read as if that administrator had personally filed the report.

Root cause: The task detail page had one code path for "Reported By" that assumed every request was reported by a real student, with no special case for a schedule-generated one.

Fix: `TaskDetail.jsx` now branches on `isPreventiveRequest(description)` and shows "Maintenance Schedule" with a plain explanation instead of a person's name for these jobs.

Verification: Manual visual check against a seeded preventive-maintenance task.

DEF-07 — Technicians received no announcements at all

Severity: Medium (a working feature that unintentionally excluded a whole role from something that plausibly benefits them, not a crash). **Found:** Transition, in response to a direct feature request that surfaced the gap: "*Maintenance Engineers should also be able to see the announcements.*"

Symptom: `GET /announcements` returned an empty list for any technician, because `announcements.service.listForUser` only had branches for `administrator` and `student`, falling through to `return []` for every other role.

Root cause: Announcements were designed and built around the student/hostel relationship first; the technician case — who is not tied to one hostel and can be sent to any of them — was never designed for in the original implementation.

Fix: Added a `selectForTechnician` branch that returns every active announcement regardless of hostel, since a technician can be assigned anywhere.

Verification: New unit tests for `selectForTechnician` and a new integration test asserting a technician token receives every active announcement regardless of its target hostel.

DEF-08 — A recurring-maintenance job's description doubly stated it was preventive

Severity: Low (cosmetic redundancy, introduced by DEF-07's neighbouring feature work, not a functional defect). **Found:** Transition, self-identified while adding the "Recurring Maintenance" badge (the same change that fixed DEF-06).

Symptom: Once a "Recurring Maintenance" badge was added to a preventive job's card, its description still read "*Preventive maintenance: Inspect geysers...*" underneath the badge — the same fact stated twice, in two different ways, on the same card.

Root cause: The description-prefix convention (`utils/preventive.js`'s `PREVENTIVE_PREFIX`) exists so the **server** can detect a preventive request without a dedicated database column; nothing about that convention

requires the raw prefix to also be shown to a user once a purpose-built badge exists to say the same thing more clearly.

Fix: Added `displayDescription()` (client-side, display-only — the stored description is untouched) to strip the prefix wherever the new badge is also shown.

Verification: Manual visual check across the dashboard, My Tasks and task detail views.

Summary

All 8 logged defects are **fixed and verified**; none remain open. Two (DEF-01, DEF-04) were High severity and both were genuine production behaviour defects rather than cosmetic issues — both are now covered by either an integration test (DEF-01, indirectly, via the request-read tests that exercise the same embed query) or a manual regression check documented above (DEF-04, which has no automated client-side test since this project has no client unit-test suite — see TESTING.md §7). The recurring theme across DEF-03/04/05/06/08 is **display and messaging defects surfacing during the Transition phase's documentation and polish work**, which is exactly the kind of defect UP expects Transition to catch — they are precisely the class of issue that only becomes visible once a feature is used end-to-end rather than unit-tested in isolation.